



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Experiment No. 7
Implement Particle Swarm Optimization
Date of Performance:
Date of Submission:



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Aim: To implement the Particle Swarm Optimization (PSO) algorithm.

Theory:

Particle Swarm Optimization (PSO) is inspired by social behavior in swarms (e.g., birds). It optimizes a function by having a population (swarm) of candidate solutions (particles) move through the search space.

The update rule for Particle Swarm Optimization (PSO) consists of updating each particle's velocity and position in the search space based on its own best-known position and the swarm's best-known position. The equations are as follows:

Velocity Update:

$$v_i^{(t+1)} = \omega v_i^{(t)} + c_1 r_1 (p_i - x_i^{(t)}) + c_2 r_2 (g - x_i^{(t)})$$

Where:

$v_i^{(t+1)}$ is the updated velocity of particle i at iteration $t+1$.

- ω is the inertia weight, which controls the influence of the previous velocity.
- c_1 and c_2 are acceleration coefficients that control the importance of personal and global best positions.
- $r_1, r_2 \sim U(0,1)$ are random numbers sampled from a uniform distribution.
- p_i is the personal best position of particle i .
- g is the global best position found by the swarm.
- $x_i^{(t)}$ is the current position of particle i .

Position Update:

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)}$$

Where: $x_i^{(t+1)}$

- $x_i^{(t+1)}$ is the updated position of the particle.

These update rules allow the particles to balance exploration (searching new regions) and exploitation (refining known good solutions) to find the optimal solution.

Steps:



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

2024-2025

1. Initialize particles with random positions and velocities.
2. Update particle velocity and position based on individual and global best solutions.
3. Continue until convergence or maximum iterations.

Procedure/Algorithm:

1. Define the optimization problem.
2. Initialize a swarm of particles with random positions and velocities.
3. Set parameters for inertia, cognitive, and social components.
4. Repeat until convergence:
 - o Update each particle's velocity based on its best position and the global best position.
 - o Update each particle's position.
5. Display the best solution found.

Code:

```
import numpy as np
import pandas as pd

class Particle:
    def __init__(self, position, min_x, max_x):
        self.position = position
        self.velocity = 0 # Initial velocity set to 0
        self.best_pos = self.position
        self.fitness = self.evaluate()
        self.best_fitness = self.fitness
        self.min_x = min_x
        self.max_x = max_x
    def evaluate(self):
        return self.position ** 2 # Fitness function f(x) = x^2
    def update_velocity(self, best_pos_swarm, w, c1, c2):
        r1, r2 = np.random.rand(), np.random.rand()
        self.velocity = (
            w * self.velocity +
            c1 * r1 * (self.best_pos - self.position) +
            c2 * r2 * (best_pos_swarm - self.position)
        )
    def update_position(self):
        self.position += self.velocity
```



Vidyavardhini's College of Engineering & Technology

Department of Computer Engineering

2024-2025

```
self.position = np.clip(self.position, self.min_x, self.max_x) # Keep within bounds
self.fitness = self.evaluate()
if self.fitness < self.best_fitness:
    self.best_fitness = self.fitness
    self.best_pos = self.position
def pso(initial_positions, max_iter, w, c1, c2, min_x=-5, max_x=5):
    particles = [Particle(pos, min_x, max_x) for pos in initial_positions]
    best_pos_swarm = min(particles, key=lambda p: p.fitness).best_pos
    best_fitness_swarm = min(p.fitness for p in particles)
    results = []
    for iter_num in range(max_iter):
        iteration_data = []
        for i, particle in enumerate(particles):
            particle.update_velocity(best_pos_swarm, w, c1, c2)
            particle.update_position()
            if particle.fitness < best_fitness_swarm:
                best_fitness_swarm = particle.fitness
                best_pos_swarm = particle.position
            iteration_data.append([i + 1, particle.position, particle.velocity, particle.best_pos, best_pos_swarm])
        df = pd.DataFrame(iteration_data, columns=["Instance", "X (Position)", "Velocity", "PBest", "GBest"])
        results.append((f"Iteration {iter_num + 1}", df))
    for title, table in results:
        print(title)
        print(table)
        print("\n")
    print("Optimal Solution:")
    print(f"GBest Position: {best_pos_swarm}, GBest Fitness: {best_fitness_swarm}")
    return results
# Example Usage
initial_positions = [1, 2, 3] # Given initial positions
max_iter = 10 # 10 iterations
w, c1, c2 = 0.5, 1.5, 1.5 # Hyperparameters
df_results = pso(initial_positions, max_iter, w, c1, c2)
```



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Output:

Iteration 1

	Instance	X (Position)	Velocity	PBest	GBest
0	1	1.000000	0.000000	1.000000	1.000000
1	2	0.776889	-1.223111	0.776889	0.776889
2	3	1.132978	-1.867022	1.132978	0.776889

Iteration 2

	Instance	X (Position)	Velocity	PBest	GBest
0	1	0.842791	-0.157209	0.842791	0.776889
1	2	0.165333	-0.611556	0.165333	0.165333
2	3	-0.637179	-1.770157	-0.637179	0.165333

Iteration 3

	Instance	X (Position)	Velocity	PBest	GBest
0	1	0.214127	-0.628664	0.214127	0.165333
1	2	-0.140445	-0.305778	-0.140445	-0.140445
2	3	-1.456219	-0.819040	-0.637179	-0.140445

Iteration 4

	Instance	X (Position)	Velocity	PBest	GBest
0	1	-0.561821	-0.775948	0.214127	-0.140445
1	2	-0.293333	-0.152889	-0.140445	-0.140445
2	3	-0.199816	1.256402	-0.199816	-0.140445

Iteration 5

	Instance	X (Position)	Velocity	PBest	GBest
0	1	0.105624	0.667445	0.105624	0.105624
1	2	-0.213149	0.080184	-0.140445	0.105624
2	3	0.678331	0.878148	-0.199816	0.105624

Iteration 6

	Instance	X (Position)	Velocity	PBest	GBest
0	1	0.439347	0.333723	0.105624	0.105624
1	2	0.180303	0.393452	-0.140445	0.105624
2	3	-0.477144	-1.155475	-0.199816	0.105624



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Iteration 7

	Instance	X (Position)	Velocity	PBest	GBest
0	1	0.098962	-0.340385	0.098962	0.098962
1	2	0.298832	0.118529	-0.140445	0.098962
2	3	-0.246886	0.230258	-0.199816	0.098962

Iteration 8

	Instance	X (Position)	Velocity	PBest	GBest
0	1	-0.071230	-0.170192	-0.071230	-0.07123
1	2	-0.749888	-1.048719	-0.140445	-0.07123
2	3	-0.085520	0.161366	-0.085520	-0.07123

Iteration 9

	Instance	X (Position)	Velocity	PBest	GBest
0	1	-0.156327	-0.085096	-0.071230	-0.071230
1	2	-0.661071	0.088817	-0.140445	-0.071230
2	3	0.003207	0.088726	0.003207	0.003207

Iteration 10

	Instance	X (Position)	Velocity	PBest	GBest
0	1	0.004381	0.160708	0.004381	0.003207
1	2	-0.161985	0.499086	-0.140445	0.003207
2	3	0.047570	0.044363	0.003207	0.003207

Optimal Solution:

GBest Position: 0.003206525499873561, GBest Fitness: 1.028180578133939e-05



Vidyavardhini's College of Engineering & Technology
Department of Computer Engineering
2024-2025

Conclusion: Particle Swarm Optimization (PSO) efficiently finds the optimal solution by simulating swarm intelligence through iterative position and velocity updates

Answer the Following Question:

1. What is the role of the inertia component in PSO?

The inertia component in Particle Swarm Optimization (PSO) helps control how much a particle's previous velocity influences its new velocity. It acts like momentum, allowing particles to keep moving in the same direction unless influenced by other forces. A high inertia value makes particles explore more of the search space, which helps find new solutions. A low inertia value makes particles focus more on refining the best solutions found so far. By balancing exploration and exploitation, the inertia component ensures that the algorithm does not get stuck too early and can find the best possible solution efficiently.

2. How does PSO differ from Genetic Algorithms?

Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) are both optimization techniques, but they work differently. PSO is inspired by the movement of swarms (like birds or fish), where particles update their positions based on their own best solution and the swarm's best solution. In contrast, GA is inspired by natural evolution, using selection, crossover, and mutation to evolve better solutions over generations. PSO focuses on adjusting velocities and positions, while GA relies on genetic operations to create diversity. PSO is generally faster and simpler, while GA is better for complex problems that require diverse solutions.